

AWS EC2 Web Server Automation

Author: Ayush Nigade

Project Type: AWS Cloud Project

Platform: Amazon Web Services (AWS)

Region: United States (N. Virginia) – us-east-1

Date: August 2026

Table of Contents

1	Project Overview
2	Project Objectives
3	AWS Services Used
4	Prerequisites
5	Project Architecture
6	Implementation Steps
7	User Data Automation Script
8	Project Verification
9	Challenges Faced
10	Learning Outcomes
11	Conclusion

1. Project Overview

Introduction

This project demonstrates the automated deployment of an Apache web server on an Amazon EC2 instance using AWS User Data. Instead of manually installing and configuring the web server after launching the instance, all required tasks are automated during instance initialization through a Bash script.

The project follows AWS best practices by using a dedicated Security Group, SSH key pair authentication, and automated server configuration. The deployed website becomes accessible through the EC2 instance's public IPv4 address.

This project provides practical experience with AWS infrastructure provisioning, Linux server administration, Bash scripting, and cloud automation.

2. Project Objectives

The objectives of this project are:

- Launch an Amazon EC2 instance.
- Configure secure network access using Security Groups.
- Automate Apache installation using EC2 User Data.
- Automatically create and host a custom HTML webpage.
- Verify successful deployment using SSH and a web browser.
- Gain practical experience with AWS cloud infrastructure.
- Document the deployment process for future reference.

3. AWS Services Used

Service	Purpose
Amazon EC2	Virtual Linux Server
Amazon Linux 2023	Operating System
Security Groups	Firewall configuration
Key Pair	Secure SSH authentication
EC2 User Data	Server automation
Apache HTTP Server	Web server
Public IPv4 Address	Access website over Internet

4. Prerequisites

Before starting the deployment, the following prerequisites were required:

- AWS Account
- IAM Administrator User
- EC2 Key Pair
- Default VPC
- Internet Connectivity
- Basic Linux Knowledge
- SSH Client
- Web Browser

5. Project Architecture

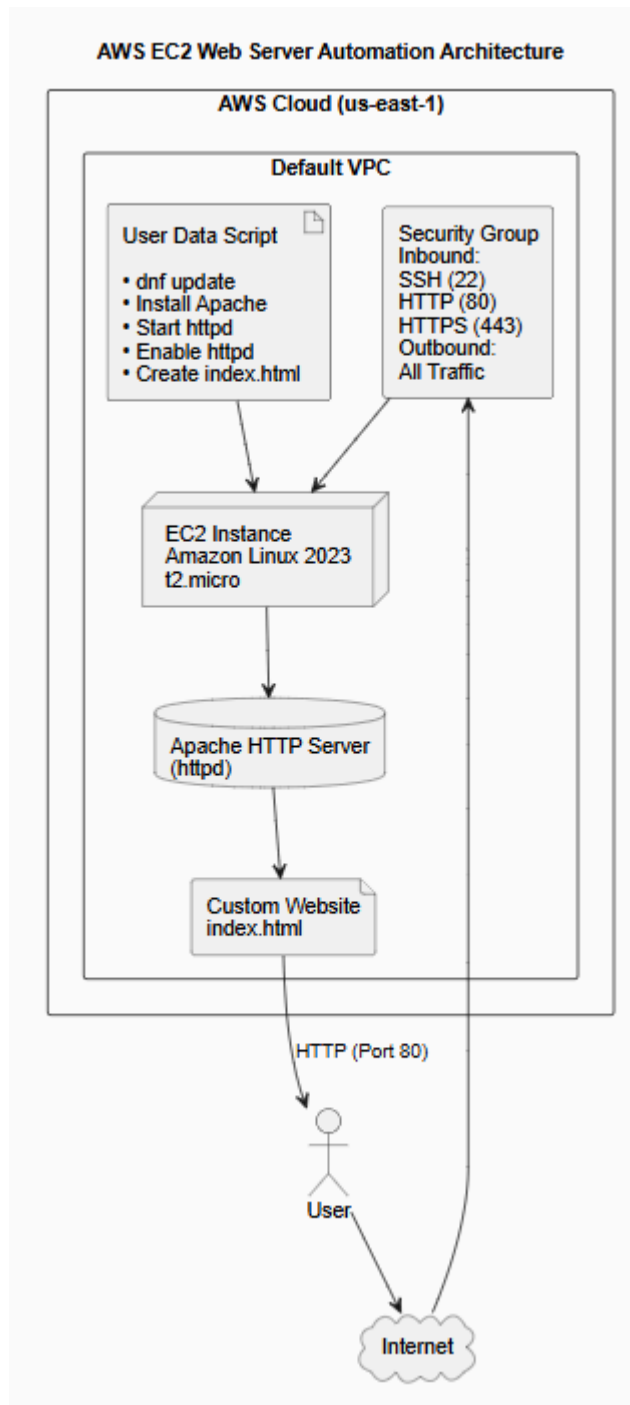


Figure 1: AWS EC2 Web Server Automation Architecture

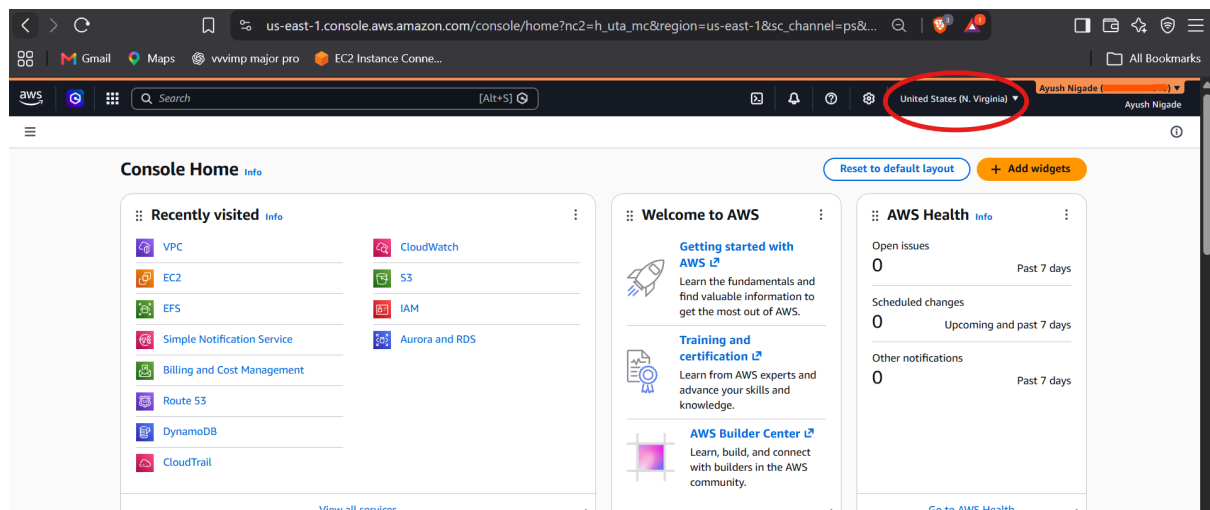
Brief Description:

The user accesses the hosted website over the Internet. Incoming traffic passes through the configured Security Group, which allows HTTP, HTTPS, and SSH access. During instance launch, the User Data script automatically installs Apache, enables the service, starts the web server, and creates a custom HTML page that is served to the user.

6. Implementation Steps

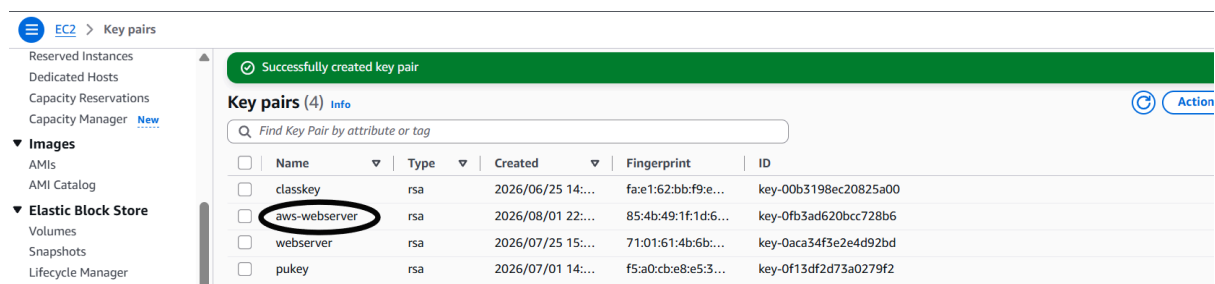
Step 1 – AWS Console Login

Logged into the AWS Management Console and selected the Mumbai (ap-south-1) region.



Step 2 – Create Key Pair

Created an RSA Key Pair named aws-webserver-key for secure SSH access.



Step 3 – Create Security Group

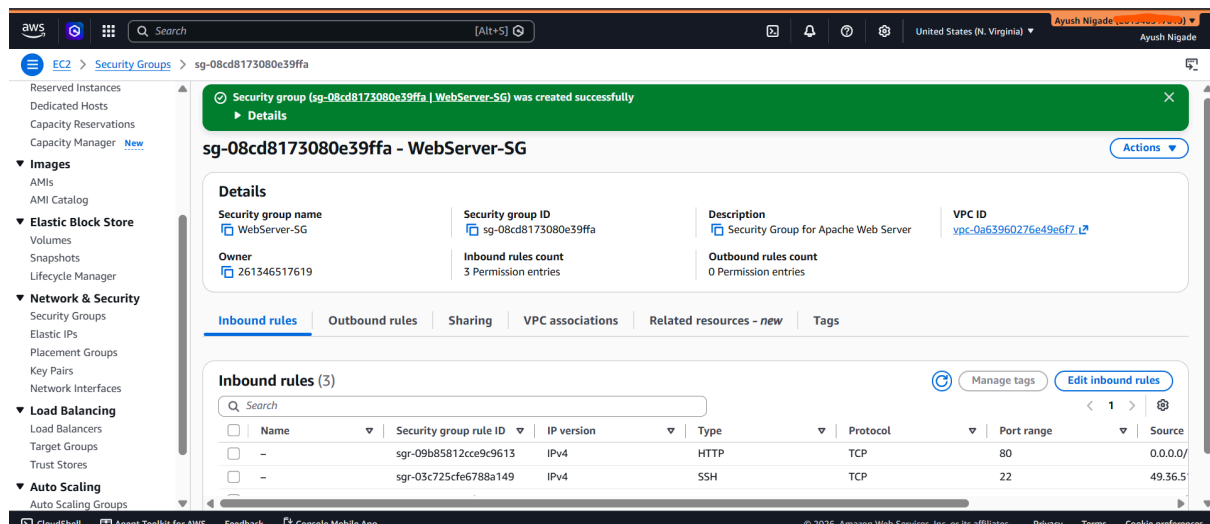
Created a Security Group with the following rules:

Inbound Rules

Port	Protocol	Source
22	SSH	My IP
80	HTTP	Anywhere
443	HTTPS	Anywhere

Outbound Rules

Port	Destination
All Traffic	0.0.0.0/0



Step 4 – Launch EC2 Instance

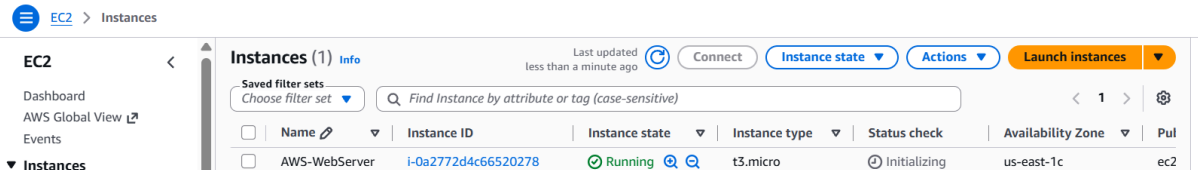
Configured the EC2 instance using:

- Amazon Linux 2023
- t2.micro
- Existing Key Pair
- Existing Security Group
- User Data Automation Script

Step 5 – Verify Instance Status

Verified that:

- Instance State = Running
- Status Checks = 2/2 Passed

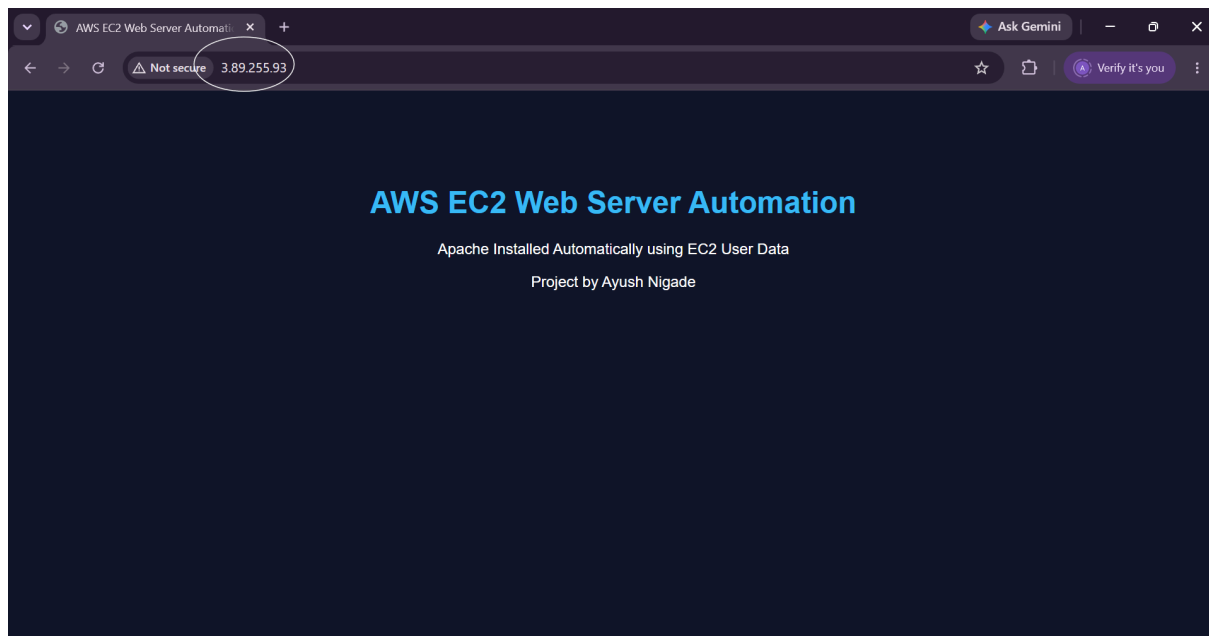
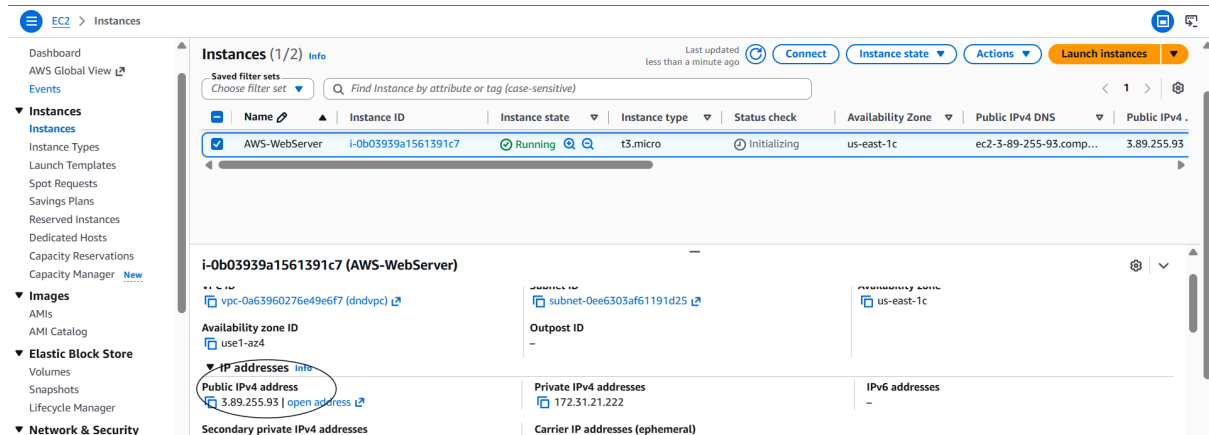


The screenshot displays the AWS Management Console for the EC2 service, specifically the 'Instances' page. The left-hand navigation pane shows the 'EC2' menu with options for 'Dashboard', 'AWS Global View', 'Events', and 'Instances'. The main content area is titled 'Instances (1)' and includes a search bar, a 'Connect' button, and buttons for 'Instance state', 'Actions', and 'Launch instances'. Below these controls is a table listing the instance details.

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Pul
☐	AWS-WebServer	i-0a2772d4c66520278	Running	t3.micro	Initializing	us-east-1c	ec2

Step 6 – Verify Website

Opened the Public IPv4 address in a web browser and confirmed that the Apache web page created by the User Data script was successfully displayed.



Step 7 – Verify Apache Service

Connected to the instance using SSH.

Executed:

```
systemctl status httpd
systemctl is-enabled httpd
cat /var/www/html/index.html
```

Confirmed that:

- Apache service is active.
- Apache is enabled on boot.
- HTML page was successfully created.

```
C:\Users\shweta>cd Downloads

C:\Users\shweta\Downloads>ssh -i "aws-webserver.pem" ec2-user@ec2-3-89-255-93.compute-1.amazonaws.com
The authenticity of host 'ec2-3-89-255-93.compute-1.amazonaws.com (3.89.255.93)' can't be established.
ED25519 key fingerprint is SHA256:ZjUDKduDMv7KxIwNXgtFBN0K4+sL7XHXChfxdxInof8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-3-89-255-93.compute-1.amazonaws.com' (ED25519) to the list of known hosts.

      #_
     _###_
    _###_#_
   _###_#_
  _###_#_
 _###_#_
V~'  _--> https://aws.amazon.com/linux/amazon-linux-2023
      /
     /
    /
   /
  /
 /
/_/

[ec2-user@ip-172-31-21-222 ~]$ sudo -i
[root@ip-172-31-21-222 ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-08-01 17:34:31 UTC; 1min 1s ago
     Docs: man:httpd.service(8)
   Main PID: 3490 (httpd)
   Status: "Total requests: 2; Idle/Busy workers 100/0; Requests served 2/0"
   Tasks: 177 (limit: 1059)
   Memory: 14.2M
     CPU: 446ms
   CGroup: /system.slice/httpd.service
```

```
root@ip-172-31-21-222:~  
[root@ip-172-31-21-222 ~]# cat /var/www/html/index.html  
<!DOCTYPE html>  
<html>  
<head>  
<title>AWS EC2 Web Server Automation</title>  
<style>  
body{  
font-family:Arial;  
background:#0f172a;  
color:white;  
text-align:center;  
margin-top:100px;  
}  
h1{  
color:#38bdf8;  
}  
</style>  
</head>  
<body>  
  
<h1>AWS EC2 Web Server Automation</h1>  
  
<p>Apache Installed Automatically using EC2 User Data</p>  
  
<p>Project by Ayush Nigade</p>  
  
</body>  
</html>  
[root@ip-172-31-21-222 ~]#
```

Step 8 – Verify User Data Execution

Verified successful execution of the User Data script by reviewing:

/var/log/cloud-init-output.log

```
Installed:  
  apr-1.7.5-1.amzn2023.0.4.x86_64  
  apr-util-1.6.3-1.amzn2023.0.2.x86_64  
  apr-util-ldb-1.6.3-1.amzn2023.0.2.x86_64  
  apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64  
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  
  httpd-2.4.68-1.amzn2023.0.1.x86_64  
  httpd-core-2.4.68-1.amzn2023.0.1.x86_64  
  httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch  
  httpd-tools-2.4.68-1.amzn2023.0.1.x86_64  
  libbrotli-1.0.9-4.amzn2023.0.2.x86_64  
  mailcap-2.1.49-3.amzn2023.0.3.noarch  
  mod_http2-2.0.42-1.amzn2023.0.1.x86_64  
  mod_lua-2.4.68-1.amzn2023.0.1.x86_64  
  
Complete!  
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /u  
Cloud-init v. 22.2.2 finished at Sat, 01 Aug 2026 17:34:31 +0000. Datasource D  
[root@ip-172-31-21-222 ~]#  
[root@ip-172-31-21-222 ~]# cat /var/log/cloud-init-output.log
```

7. User Data Automation Script

Insert your complete Bash script here.

Brief Description:

The User Data script automatically performs the following operations during the first boot:

- Updates the operating system.
- Installs Apache HTTP Server.
- Starts the Apache service.
- Enables Apache to start automatically on boot.
- Creates a custom HTML homepage.

This eliminates manual server configuration and demonstrates infrastructure automation using EC2 User Data.

8. Project Verification

The project was verified using the following checks:

- EC2 instance successfully launched.
- Status checks completed successfully.
- Apache service running.
- Apache service enabled at boot.
- Custom HTML page displayed in browser.
- User Data executed successfully.
- SSH connectivity verified.

Project Status:

Deployment Successful

9. Challenges Faced

During implementation, the following practical challenges were encountered and resolved:

- Configuring Security Group rules correctly.
- Waiting for EC2 initialization before testing the website.
- Understanding the execution flow of EC2 User Data.
- Verifying cloud-init logs to confirm automation.
- Managing SSH key permissions for secure login.

10. Learning Outcomes

Through this project, I gained practical experience in:

- AWS EC2 provisioning.
- Linux server administration.
- Apache Web Server deployment.
- EC2 User Data automation.
- Security Group configuration.
- SSH connectivity.
- Cloud infrastructure deployment.
- Bash scripting.
- Technical documentation.
- GitHub project management.

11. Conclusion

This project successfully demonstrated the automated deployment of an Apache web server on AWS using Amazon EC2 and User Data automation. The implementation reduced manual configuration by executing all required setup tasks during instance initialization. The project provided hands-on experience with cloud infrastructure, Linux administration, and automation techniques that are commonly used in real-world DevOps and Cloud Engineering environments.